

基于 Gate 隐藏状态信号的 Steering 方案调研

摘要

现有 gate（无论 Δh only、 h only 还是特征融合版本）本质上是一个只读（read-only）的判分器：它消费 Llama 的隐藏状态，输出“这一跳证据选择是否异常”的概率，下游只用这个概率去调整候选段落的排序（ $\text{emb_score} - \lambda \cdot \text{gate_score}$ ），从不反过来修改 Llama 自身的计算过程。Steering（激活/表征层面的干预）想做的是反过来：把 gate 已经学到的“异常方向”直接写回模型的隐藏状态，在生成过程中主动把模型的内部状态从“异常”推向“正常”，而不仅仅是在事后重排序候选。

本报告调研 steering 相关技术的方法谱系、评估其中哪些能够零成本复用本项目已有的训练产出（gate 的 PCA+LR 权重、多层隐藏状态提取管线），并给出按工程复杂度分级的落地路径与验证方法。核心结论：**方向向量不需要重新训练**——直接从已有 gate 的 `LogisticRegression.coef_` 反投影回 4096 维隐藏空间即可得到；真正的门槛在于生成侧目前跑在 vLLM 上，而 vLLM v1 引擎不像 HF transformers 那样能直接挂 forward hook 修改中间激活，这是决定落地路径优先级的关键工程约束。

一、背景：为什么是 steering，以及它和现有 gate 的关系

1.1 现有 gate 的干预方式是“外部重排序”

第六节（`gate_v3_main_method_report.md`）描述的推理流程里，gate 只做一件事：给每个候选证据打一个“该证据是否异常”的连续分数，参与 $\text{final_score} = \text{emb_score} - \lambda \cdot \text{gate_score}$ 排序。这个分数从未被用来改变 Llama 自身后续的计算——模型看到的候选证据文本、生成的简短答案、下一跳的子问题，都不知道“gate 刚才判断这里有问题”这件事。

1.2 Steering 想解决的问题

如果 gate 在某一跳判断“当前累积状态明显异常”（即使候选池里没有更好的选项，或者异常来自更早的跳而非当前候选本身），单纯重排序无能为力——重排序只能在给定候选池内选择相对更好的一个，无法让模型“意识到”自己的推理链路可能已经跑偏、进而调整后续生成（比如更谨慎地生成下一跳子问题、或在最终答案生成时降低对某个中间结论的置信度）。Steering 直接操作隐藏状态，理论上可以影响生成内容本身，而不仅是候选排序。

1.3 关键的表征假设

Steering 依赖“线性表征假设”（linear representation hypothesis）：许多高层概念在模型的隐藏状态空间中以线性方向的形式编码，即“沿某个方向平移隐藏状态”就能连续地增强或减弱某个概念的表达。本项目的 gate 本身就是这个假设的一个直接验证：PCA+LR 能在几万样本上以 0.92+ AUC 区分“证据选择是否有误”，说明“是否异常”这个概念在 Llama 中间层的隐藏状态里，确实存在一个（在 PCA 子空间内）近似线性可分的方向。这正是 steering 能够复用 gate 现有产出的理论基础。

二、Steering 技术谱系

以下方法在 2023–2026 年的表征工程（representation engineering）文献中反复出现，构成了当前的主流框架（[Representation Engineering for LLMs: Survey and Research Challenges](#)）。

2.1 Activation Addition (ActAdd, Turner et al., 2023)

最基础的形式：用一对对比 prompt ("正面"/"负面") 在某一层的隐藏状态之差作为方向向量，推理时 把这个向量按固定系数加到该层所有 token 的激活上。优点是完全不需要训练，只需要跑两次前向传播 做差；缺点是方向来自单一对比，噪声较大，通常需要多对样本取平均。

2.2 Contrastive Activation Addition (CAA, Panickssery et al., 2023)

ActAdd 的稳健化版本：在大量正负样本对上分别取隐藏状态均值，方向 = 正例均值 - 负例均值 (difference-of-means)。这与本项目 gate 训练数据的构造方式高度吻合——gold trace (正例) 与 counterfactual trace (负例) 本来就是成对构造的，`hidden_states/pilot_multilayer/{pos,neg}/` 目录已经是现成的对比数据，理论上不需要新提取任何隐藏状态就能算出一版 CAA 方向，作为 gate LR 方向之外的第二种独立验证。

2.3 Representation Engineering (RepE, Zou et al., 2023)

比 CAA 更系统化的框架：用 PCA (而非简单均值差) 从对比样本中提取"读出方向" (reading vector)，同一方向既可用于读 (训练线性探针，即本项目的 gate) 也可用于写 (作为 steering 方向)。本项目 gate 的 PCA+LR 结构，在 RepE 的术语体系下已经是一个完整的"reading vector 提取器"——唯一缺的是把这个方向反过来用于"writing" (即 steering)。

2.4 Inference-Time Intervention (ITI, Li et al., 2023/2024)

在 attention head 粒度 (而非整个残差流) 上做 steering，先探测哪些 head 对目标概念最敏感，只干预这些 head 的输出。精度更高、副作用更小，但需要逐 head 训练探针，工程量显著高于整层残差流 steering，适合本方案验证有效后的精细化阶段，不建议作为第一版方案。

2.5 稀疏自编码器 (SAE) 特征级 steering

在模型激活上训练一个稀疏自编码器，把稠密的隐藏状态分解成大量 (通常数万到数十万) 单语义 特征，直接夹持 (clamp) 某个具体特征的激活值实现更精细、更可解释的控制 (Anthropic "Golden Gate Claude" 即此类工作的公开示例)。效果通常优于粗粒度的线性方向 steering，但需要为 Llama- 3.1-8B 单独训练一个高质量 SAE，成本 (数据、算力、调参) 远高于本项目目前的任何一步，**不建议作为第一阶段方案**，仅作为"如果线性方向 steering 验证有效但精度不够"时的后续选项列出。

2.6 自适应强度 steering (Adaptive Activation Steering / ACT, 2025)

2025 年出现的改进方向：不用固定系数把方向向量加到激活上，而是根据当前激活"偏离正常程度"的连续度量，动态调整干预强度，思路是"越异常干预越强，越正常干预越弱/不干预"。这与本项目现有设计高度吻合——gate 本身已经输出连续概率 p_j (而非仅二元触发)，完全可以直接把 steering 强度设计为 p_j 的函数 (例如 $\alpha = \max(0, p_j - \text{threshold}) \times \text{scale}$)，无需额外训练自适应 强度模块，是这次调研里与现有产出复用度最高的一个改进点。

2.7 已知局限与风险

- **方向不稳定/不可识别**：同一个"概念"用不同对比数据、不同层可能训出方向不同甚至相互矛盾的 steering vector，在训练数据分布外的输入上效果可能减弱甚至反向 ([On the Non-Identifiability of Steering Vectors in Large Language Models](#))。
- **可能引发意外的能力损失或行为偏移**：steering 强度过大时，观察到模型在与目标概念无关的任务上性能下降，甚至出现价值观/安全性层面的意外偏移 ([Activation Steering Induces Emergent Misalignment: A More Comprehensive Evaluation](#))。这意味着**必须做无关任务的 sanity check**，不能只看多跳检索本身的指标变好就判定 steering 成功。
- **细粒度优于粗粒度**：近期工作 ([Fine-Grained Activation Steering](#)) 显示，把 steering 限制在更少的层/更少的 token 位置、强度更精细地控制，效果优于"整层全 token 固定强度"的粗放式做法——与

2.4/2.6 的方向一致。

本项目所在的"steering 用于多跳检索证据选择纠错"这一具体场景，在调研中未检索到直接对应的 已发表工作，是一个相对空白的应用方向，但底层技术（对比方向提取、条件化强度）都是成熟的。

三、映射到本项目：方案设计

3.1 方向向量：可以直接复用现有 gate 产出，不需要重新训练

三种 gate 的 artifact 结构（`joblib.dump` 的字典）已经包含了构造方向向量所需的全部信息：

gate 变体	artifact 路径	字典 key	特征空间
Δh only (v2)	gate/artifacts_pooled_v2/j{0..3}.joblib	pca, lr	Δh_j (4096 维)
h only	gate/gate_h_only/artifacts_pooled_h_only/j{0..3}.joblib	pca_h, lr	h_after (4096 维)
融合 (v3)	gate/gate_v3/artifacts_pooled_v3/j{0..3}.joblib	pca_h, pca_delta, lr	h_after $\oplus \Delta h_j$

单 PCA 分支的方向反投影公式：

```
m = joblib.load("j0.joblib")
w_full = m["lr"].coef_[0] @ m["pca"].components_ # (4096,)，指向"应该触发干预"的方向
steer_dir = -w_full / np.linalg.norm(w_full) # 取反并归一化，指向"正常"方向
```

v3（拼接特征）需要按 `nc_h/nc_delta`（两个 PCA 各自保留的主成分数，训练时已记录在 `meta.json` 里）把 `lr.coef_` 切成两段，分别反投影回 h 空间和 Δh 空间——得到两个独立方向，语义不同：h 方向直接加到某跳完成后的隐藏状态上， Δh 方向表示"这一步的变化量应该往哪个方向修正"，两者虽然维度相同（都是 4096）、可以相加使用，但物理意义不同，实验时建议先分开验证再考虑 是否联合使用。

这三个方向本质上就是 2.3 节 RepE 的"reading vector"，且已经用大规模数据训练、在分类任务上 验证过有效性（AUC 0.92-0.94），比从零构造 CAA 方向（差分均值）更成熟，应作为第一版实验的主选项；CAA 差分均值可以作为独立方向来源，用于交叉验证 gate 方向是否稳健（2.7 节提到的"方向 不稳定"风险，最省成本的检验方式就是看两种独立方法算出的方向是否高度一致，如余弦相似度）。

3.2 层选择：复用现有验证结果

沿用 gate v2/v3/h-only 已经验证过的层选择（j=0,1 用第 15 层，j=2,3 用第 23 层），不需要重新 搜层——这几层已经被证明是"异常信号"线性可分性最好的层，没有理由假设 steering 需要不同的层。

3.3 工程可行性的核心瓶颈：vLLM 与 HF transformers 的分工

这是本调研中最重要的一个工程判断。当前推理管线里有两个独立的 Llama-3.1-8B-Instruct 实例：

- **gate 打分用的模型** (`AutoModelForCausalLM`, HF transformers, `run_retrieval_exp_wavefront.py` 第 330/346 行加载)：只做前向传播提取隐藏状态, 不生成文本。HF 模型天然支持 `register_forward_hook`, 在这个模型上加 steering 向量**零工程成本**——但改了它也没用, 因为它不负责生成任何文本, 改了只会改变 gate 自己的打分 (这本身是有用的验证手段, 见 3.5 节, 但不是"steering 影响生成"意义上的 steering)。
- **真正生成推理链路文本的模型** (`BatchVllmGenerator`, `from vllm import LLM`, `run_retrieval_exp_wavefront.py` 第 254/265 行)：负责候选选择、逐跳简短答案、最终答案生成, 即 steering 真正需要影响的对象。vLLM v1 引擎 (当前版本 0.19.0) 用编译后的 CUDA Graph + PagedAttention 管理 KV cache, 模型的 forward 不是一个可以用标准 PyTorch `register_forward_hook` 直接拦截的普通函数调用, 中途修改激活比 HF 模型困难得多。

也就是说, "要不要 steering、往哪个方向 steering"这个判断可以完全在已有的 HF 侧完成 (gate 本来就在这个模型上跑), 但"steering 之后真正生成不一样的文本"这个环节, 必须想办法接触到 vLLM 内部或者绕开 vLLM。

3.4 三条落地路径 (按工程复杂度从低到高)

方案 A：仅对 gate 判定风险较高的少数跳, 生成步骤从 vLLM 切换回 HF `transformers.generate()`

需要先纠正一处表述：当前主方法 (gate v3 的 rerank 流程) 里 gate 并不是"触发式"的——每一跳召回的 10 个候选会全部过一遍 gate 打分, 全部参与 `rank_key = emb_score - λ · gate_score` 重排, 这个过程对每一跳都无条件发生, 不存在"多数跳不触发所以不用管"这回事 (那是 `gated_rule_a` 这个规则触发变体的行为, 跟主方法是两套不同的干预方式, 不能混用)。`target_fpr=0.15 / threshold` 这两个数字来自 gate 训练时在**离线、正负例各半的分类 dev 集**上标定的分类阈值 (`fit_lr_gate_pooled_v3.py` 里的 `tau`), 衡量的是"gate 自己的分类器在均衡数据上的误报率", 不代表真实检索候选池 (正常情况下大多数候选池本来就没有明显异常候选) 上实际会有多大比例的跳超过这个阈值——这个比例目前没有测过, 是一个待测的经验值, 不能假设它天然很低。

方案 A 真正能省成本的地方, 是把"gate 打分" (已经在做, 本来就无条件跑) 和"steering 需要的 额外生成成本" (vLLM→HF 切换) 这两件事解耦: gate 打分不用改, 继续对 10 个候选全部打分、正常参与重排; 重排完成、确定这一跳最终选中的候选 (rank-1) 之后, **新引入**一个独立于现有重排逻辑的判断——如果这个最终选中候选的 `gate_score` 仍然偏高 (说明"即使排序后最好的候选, gate 也不放心"), 才对这一跳的后续生成 (该跳简短答案) 额外付出 HF fallback 的代价, 用 `register_forward_hook` 按 3.6 节的强度公式注入方向向量后调用 `.generate()`; `gate_score` 不高的跳, 简短答案生成继续走 vLLM, 不受影响。这个新阈值该定多高、实际会命中多大比例的跳, 需要先在已有的 `retrieval_cases` (已经记录了每跳每候选的 `gate_v3_score`) 上做一次离线统计, 不能凭空假设"是小概率事件"。

- 优点：改动面小, gate 打分部分完全不用动 (本来就在跑), 只在"重排后仍判定为风险较高"的跳上额外付出生成成本, 如果离线统计显示这个新阈值命中率确实不高, 两三天内能出第一版 pilot。
- 缺点：命中率是待测的未知数, 不能提前假设低; 命中的跳生成速度显著慢于 vLLM (无 PagedAttention/批处理), 如果命中率在某些数据集/某些 K 上偏高, 端到端耗时会明显增加; 两套生成路径 (vLLM + HF) 需要维护生成参数、prompt 模板完全一致, 否则引入新的分布不一致问题 (本项目在第八节已经因为类似的训练/推理文本分布不一致问题吃过亏, 需要特别小心复现同一个坑)。

方案 B：直接在 vLLM 内部挂 steering hook (不切后端)

vLLM 提供了 plugin 化的模型注册机制, 理论上可以自定义一个包一层 hook 的 `LlamaForCausalLM` 子类并通过 vLLM 的模型注册接口接入, 在其 `forward` 里对指定层的隐藏状态做加法后再往下传, 从而让所有生成 (不论 gate 判定风险高低) 都能被 steering 覆盖, 不需要在 vLLM/HF 两条路径之间切换。

- 优点：架构统一，不引入方案 A 的双路径分布不一致风险，长期看更干净。
- 缺点：需要熟悉 vLLM v1 内部模型注册与 CUDA Graph 编译流程，出错定位成本高于 HF hook；该项目目前没有任何 vLLM 内部改造的先例，从零摸索的时间成本明显高于方案 A。

方案 C：训练专用方向（CAA/RepE 从头构造，或引入 SAE）

如果方案 A 的 pilot 显示"gate 的 LR 方向"确实能有效改变生成（3.5 节的判据通过），但效果强度或精度不够，再考虑：(i) 用 2.2 节的 CAA 方法重新构造一版差分均值方向做对比；(ii) 引入 2.5 节的 SAE 做更精细的特征级干预。这两者都比方案 A/B 贵得多，建议放在验证出"方向本身有效"之后再启动，不作为第一阶段工作。

建议顺序：先做方案 A 的 pilot，验证"方向有效性"这个最核心的假设是否成立，再决定是否值得投入方案 B 的工程量；方案 C 仅在方案 A 证明方向本身不够精确时才需要。

3.5 最省成本的第一步：不碰生成，只验证方向能不能"骗过"gate 自己

在启动任何生成侧改动之前，有一个几乎零成本、且完全复用现有代码和数据的验证实验：

1. 取 `hidden_states/pilot_multilayer/dev/activations/neg/`（counterfactual trace，即 gate 应该判定为"异常"的负例）里已知 `wrong_hops` 的那一跳的隐藏状态。
2. 按 3.1 节公式算出的方向向量，以不同强度加到这个隐藏状态上。
3. 用同一个 gate 重新打分——如果方向和强度选得对，"加了方向之后"的分数应该系统地往"正常"方向移动（触发概率下降），且不触发跳（gold trace 的正例）加了之后分数不应显著变化（否则说明方向不够精确，把不该动的也动了）。

这个实验不需要 vLLM，不需要改任何生成代码，用现有的 `apply_lr_gate_pooled.py` 打分逻辑稍加改造即可，几个小时内能出结果，且直接回答了"gate 学到的方向是否真的对应一个可操作的、线性的'异常→正常'方向"这个最基础的问题——如果这一步都不成立，后面生成侧的复杂工程投入就没有意义。

3.6 自适应强度：复用 gate 已有的连续分数

按 2.6 节的思路，steering 强度不用固定系数，直接用 gate 本来就会算的概率 p_j （具体取"这一跳 经过重排后最终选中的候选"的 `gate_score`，而非 10 个候选各自的分数——steering 作用的是这一跳 累积下来的隐藏状态，需要一个跳粒度而非候选粒度的判断）：

```
 $\alpha_j = \text{scale} \times \max(0, p_j - \text{threshold})$ 
 $h_{\text{steered}} = h + \alpha_j \times \text{steer\_dir}$ 
```

`threshold` 可以先拿 gate artifact 里已经存的分类阈值（`tau`）当起点，但如 3.4 节所述，这个阈值是在离线均衡分类数据上标定的，不代表它在真实候选池分布上同样对应"少数跳命中"，用于 steering 前需要先用离线统计重新核实、可能需要单独标定一版。 p_j 越高（gate 越确信这一跳 异常），干预越强； p_j 低于阈值时 $\alpha_j = 0$ ，完全不干预，与方案 A"只对 gate 判定风险较高的跳额外付出生成成本"的设计吻合，`scale` 是唯一需要新引入、需要扫描的超参数。

四、验证方法与风险检查

4.1 分阶段验证

阶段	验证内容	是否需要生成/vLLM	复用现有代码
1	方向能否让 gate 自己改判 (3.5 节)	否	<code>apply_lr_gate_pooled.py</code>
2	方向来源交叉验证 (gate LR 方向 vs. CAA 差分均值方向, 余弦相似度)	否	复用 <code>hidden_states/pilot_multilayer/{pos,neg}</code>
3	小样本 (<code>--limit 50~100</code>) 生成侧 pilot : steering 开/关 对话短答案质量、下一跳子问题的直接对比	是 (方案 A)	<code>run_retrieval_exp_wavefront.py</code> 改造
4	端到端 recall@1/@3、chain、EM、F1 对比 (steering vs. 现有纯重排序方案)	是	同上, 全量 dev
5	无关能力 sanity check (见 4.2)	是	需要一个和多跳检索无关的通用评测集

4.2 必须做的风险检查

按 2.7 节列出的两个已知风险，落地到本项目：

- **强度扫描要看是否单调**： α 从 0 逐步增大，gate 触发率/端到端指标应该平滑变化，如果在某个 强度之后突然反转或指标崩溃（未训练层面出现的非线性），说明方向或强度公式有问题，不能只跑 一个固定强度就下结论。
- **无关能力探测**：由于 steering 是在残差流层面全局修改隐藏状态，即使只在触发的跳生效，也 可能影响该跳之后模型的其他能力（比如简短答案的流畅度、格式遵循）。建议至少做一个简单的 "格式是否仍然合法"（能否被下游解析逻辑正确读取）和"该跳未涉及的其他事实性内容是否被误改" 的抽样人工检查，而不是只看 EM/F1 数字变好就认为万事大吉。

五、优先级与路线图建议

1. **第一步（几小时，零 GPU 生成成本）**：3.5 节的"方向能否让 gate 自判改变"实验，附带 3.2 节提到的 CAA 交叉验证。这是判断整个方向是否值得投入的第一道门槛。
2. **第二步（如果第一步通过，约 2-3 天）**：方案 A 的 HF fallback pilot, `--limit` 小样本，验证生成侧确实发生了预期方向的变化，同时按 4.2 节做基础的风险检查。
3. **第三步（如果第二步显示端到端指标有正向收益）**：全量 dev 三个数据集的方案 A 端到端对比，与现有 gate v3 主方法（重排序）叠加或替代对比，产出可以直接放进 `gate_v3_main_method_report.md` 的新一节实验结果。
4. **第四步（可选，视第三步收益大小决定是否投入）**：方案 B（vLLM 内部 hook，工程量显著更大）或方案 C（SAE/专用 CAA 方向，成本更高），只有在方案 A 已经证明"steering 这条路本身有效"之后才值得投入。

参考文献

- Turner et al., 2023. *Steering Language Models With Activation Addition (ActAdd)*.

- Panickssery et al., 2023. *Steering Llama 2 via Contrastive Activation Addition* (CAA).
- Zou et al., 2023. *Representation Engineering: A Top-Down Approach to AI Transparency* (RepE).
- Li et al., 2023/2024. *Inference-Time Intervention: Eliciting Truthful Answers from a Language Model* (ITI).
- [Representation Engineering for Large-Language Models: Survey and Research Challenges](#)
- [On the Non-Identifiability of Steering Vectors in Large Language Models](#)
- [Activation Steering Induces Emergent Misalignment: A More Comprehensive Evaluation](#)
- [Fine-Grained Activation Steering: Steering Less, Achieving More](#)
- [Patterns and Mechanisms of Contrastive Activation Engineering](#)
- Anthropic, *Towards Monosemanticity / Scaling Monosemanticity* (稀疏自编码器特征级 steering 的代表性公开工作)。